

FINPILOT AI

Complete Technical Specification & Build Document

Project Specification for Development (Cursor-Ready)

Contents

1	1. Executive Summary	3
1.1	1.1 What Problem Does FINPILOT Solve?	3
1.2	1.2 Core Use Cases	3
1.3	1.3 What Makes This Different From a Normal Web App Build	4
2	2. Project Goals & Success Criteria	5
2.1	2.1 Goals	5
2.2	2.2 Success Criteria (Definition of Done per Stage)	5
3	3. High-Level Architecture	6
3.1	3.1 Architectural Principles	6
4	4. Complete Technology Stack	7
4.1	4.1 Stage 1 — Foundation Technologies	7
4.2	4.2 Stage 2 — Intelligence Technologies	7
4.3	4.3 Stage 3 — Personal CFO Technologies	8
4.4	4.4 Cross-Cutting / DevOps (applies from Stage 2 onward)	8
5	5. Detailed Feature Specification	9
5.1	5.1 Stage 1 Features (Foundation)	9
5.1.1	5.1.1 Landing Page	9
5.1.2	5.1.2 Authentication UI (frontend-only prototype at this stage)	9
5.1.3	5.1.3 Dashboard	9
5.1.4	5.1.4 Income Management	9
5.1.5	5.1.5 Expense Management	9
5.1.6	5.1.6 Loan/EMI Management	9
5.1.7	5.1.7 Financial Calculations (pure JS functions)	9
5.1.8	5.1.8 Data Model (Stage 1 — client-side only)	9
5.2	5.2 Stage 2 Features (Intelligence)	10
5.2.1	5.2.1 Real Authentication	10
5.2.2	5.2.2 Database Schema (PostgreSQL via Prisma)	10
5.2.3	5.2.3 REST API Specification	11
5.2.4	5.2.4 AI Financial Chat — Data-Grounding Pattern	12
5.2.5	5.2.5 Spending Pattern Analytics	13
5.2.6	5.2.6 External Financial Data Integration	13
5.3	5.3 Stage 3 Features (AI Personal CFO)	13
5.3.1	5.3.1 AI Agent Architecture	13
5.3.2	5.3.2 Tool Definitions (function-calling schema, conceptual)	13
5.3.3	5.3.3 Goal Planning Module	14
5.3.4	5.3.4 Purchase Planner	14
5.3.5	5.3.5 Vacation Planner	14
5.3.6	5.3.6 Credit-Trap Prevention	14

5.3.7	5.3.7 Investment Portfolio & Analysis	14
5.3.8	5.3.8 Scenario Engine	14
5.3.9	5.3.9 Retirement Planner	15
5.3.105.3.10	Continuous AI Insights (background job)	15
6	6. Non-Functional Requirements	16
7	7. Repository & Project Structure (Recommended)	17
8	8. Build Roadmap & Milestones	18
8.0.1	8.1 Suggested Two-Week Sprints (Stage 1 example)	18
9	9. Instructions for AI-Assisted Development (e.g., Cursor)	19
10	10. Appendix — Glossary of Key Terms	20

1. Executive Summary

FINPILOT AI is a full-stack, AI-driven personal finance web application that evolves from a simple financial tracker into a complete **AI Personal CFO (Chief Financial Officer)**. It helps a user record their financial life (income, expenses, loans), understand it (analytics, trends, spending intelligence), and plan it (goals, investments, scenarios, retirement) — using a real backend, a real database, external financial data, and a tool-calling AI agent.

This document is a **complete build specification**. It is written so that it can be handed directly to an AI coding assistant (e.g. Cursor, Claude Code, GitHub Copilot Workspace) along with the codebase, and used to scaffold, implement, and iterate on the entire application — frontend, backend, database, AI layer, and DevOps — end to end.

1.1 What Problem Does FINPILOT Solve?

Most personal finance apps fall into one of two weak categories:

1. **Pure trackers** (e.g. spreadsheet-style expense loggers) — they record data but provide no intelligence, no forward planning, and no personalized guidance.
2. **Generic robo-advisors** — they give generic advice based on broad demographic assumptions, not the user's actual, real-time financial behavior.

FINPILOT's differentiation: it combines (a) granular personal financial data, (b) real external market/economic data, and (c) an AI agent with structured “tools” (functions) to compute real numbers — so the AI never guesses or hallucinates financial figures. Every AI response is grounded in a calculation the backend actually performed.

1.2 Core Use Cases

#	Use Case	Example
1	Track income & expenses	“I earn Rs. 80,000/month; I spent Rs. 9,000 on food this month.”
2	Understand spending behavior	“Why did my savings drop this month compared to last month?”
3	Manage loans/EMIs	“Track my car loan of Rs. 8,00,000 at 9.5% interest with EMI Rs. 17,000.”
4	Get proactive financial insights	“Your food spending increased 25% over your 3-month average.”
5	Plan a major purchase	“I want to buy a car worth Rs. 12,00,000 in 3 years — can I afford it?”
6	Avoid over-leveraging	“This new EMI would push your debt-to-income ratio above a safe threshold.”
7	Analyze an investment portfolio	“Your allocation is 55% equity, 25% debt, 10% gold, 10% cash — here's the risk profile.”
8	Model financial scenarios	“At Rs. 10,000/month for 10 years, here are conservative/base/optimistic outcomes.”
9	Plan for retirement	“At your current savings rate, you'll reach Rs. X corpus by age 60 — the gap to your goal is Rs. Y.”
10	Chat naturally with an AI CFO	“Explain my current financial health in plain language.”

1.3 1.3 What Makes This Different From a Normal Web App Build

Typical CRUD App	FINPILOT AI
Stores and displays data	Stores, analyzes , and acts on data
Static dashboards	Dashboards and an AI agent that reasons over the same data
No external data	Ingests external market/economic data via APIs
AI (if any) is a decorative chatbot	AI is tool-calling , grounded in real calculations, never a black box
Single-stage build	Three explicit stages — Foundation → Intelligence → Personal CFO — each independently demoable
One large monolith from day 1	Intentionally staged architecture: LocalStorage prototype → real backend/DB → AI agent layer

2. Project Goals & Success Criteria

2.1 Goals

- G1: Ship a fully working financial dashboard (income, expenses, loans, savings, charts) usable stand-alone, with zero AI dependency.
- G2: Migrate that prototype to a real, secure, multi-user backend with persistent storage.
- G3: Layer in external financial/market data and a first AI intelligence layer (chat, savings insight).
- G4: Build a tool-calling AI agent capable of goal planning, investment analysis, scenario modelling, and proactive insights.
- G5: Ship a responsive, accessible, production-grade UI throughout every stage.

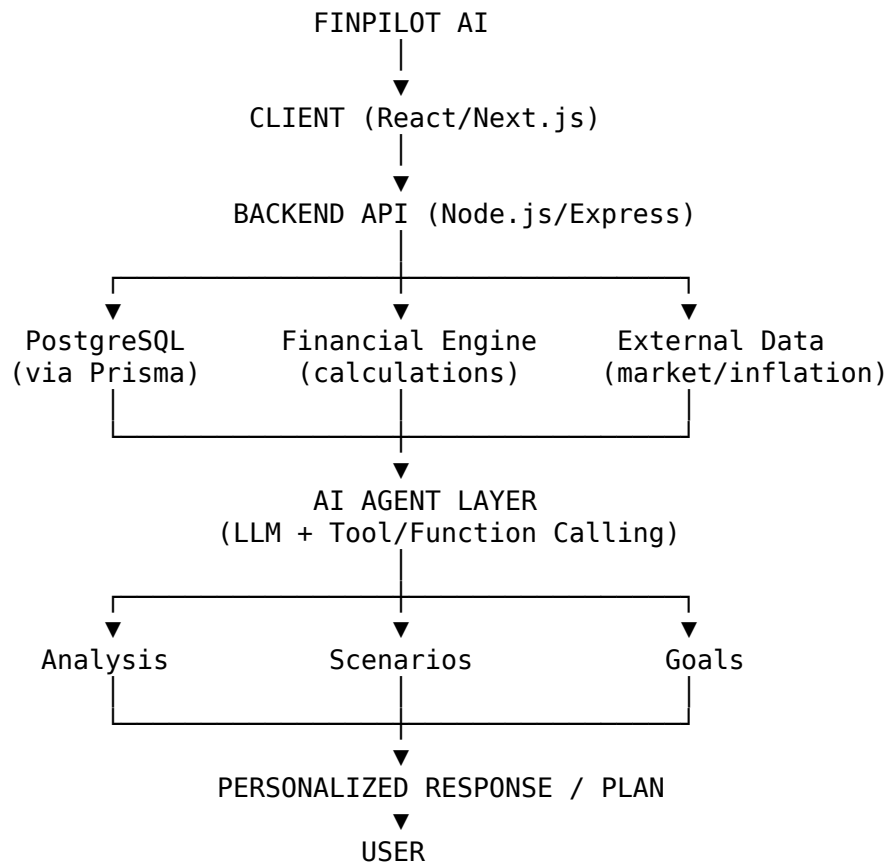
2.2 Success Criteria (Definition of Done per Stage)

Stage 1 — Foundation is done when: - A user can add/edit/delete income, expenses, and loans through forms. - Dashboard displays live totals: income, expenses, savings, savings rate, loan balance. - At least 3 charts render correctly (income vs expense, expense breakdown, savings trend). - All data persists in localStorage across refresh. - UI is responsive across desktop, tablet, and mobile breakpoints.

Stage 2 — Intelligence is done when: - Users can register/login with hashed passwords and session/JWT-based auth. - All CRUD operations for income/expense/loans go through REST APIs backed by PostgreSQL. - A user can only ever see and modify their own data (row-level authorization enforced server-side). - At least one external financial data source is integrated and cached. - The AI can answer natural-language questions about the user's own data (e.g., "why did my savings drop") using real backend-computed figures, not guesses.

Stage 3 — Personal CFO is done when: - An AI agent can call backend "tools" (functions) to fetch data and perform calculations before responding. - A user can create a financial goal (purchase, vacation, retirement) and get a computed plan with a savings gap. - Investment portfolio entries can be added and analyzed (allocation, risk commentary). - Scenario modelling produces at least 3 projections (conservative/base/optimistic) for any recurring investment. - The system can generate at least one type of proactive insight automatically (e.g., "expenses increased X% vs 3-month average").

3. High-Level Architecture



3.1 Architectural Principles

1. **Progressive complexity.** No technology is introduced before there's a concrete reason for it (no backend until Stage 2, no AI until it has real data to reason over).
2. **AI never invents numbers.** The AI layer only receives values already computed by the backend's financial engine — this is enforced via tool/function calling, not by asking the AI nicely.
3. **Multi-tenant by design from Stage 2 onward.** Every table, every query, every API route assumes multiple users and enforces per-user data isolation at the database and API layer (not just in the UI).
4. **Stateless API, stateful database.** The backend API itself holds no session state in memory beyond what's needed for the request; all durable state lives in PostgreSQL (and, for caching, Redis).
5. **Everything is testable in isolation.** The financial engine (pure calculation functions) must be unit-testable without a database, a server, or an AI model.

4. Complete Technology Stack

4.1 Stage 1 — Foundation Technologies

Layer	Technology	Purpose
Structure	HTML5	Page structure, semantic markup, forms
Styling	CSS3 (or Tailwind CSS)	Dashboard, sidebar, responsive layout
Logic	JavaScript (ES6+)	Form handling, CRUD logic, calculations
Charts	Chart.js	Income/expense, breakdown, savings trend charts
Persistence	Browser localStorage	Prototype data persistence (pre-backend)
Tooling	Git + GitHub	Version control, commit history, code review
Editor	VS Code	Development environment

No AI, no backend, no database in Stage 1 — by design. This proves the complete financial workflow end-to-end before adding infrastructure complexity.

4.2 Stage 2 — Intelligence Technologies

Layer	Technology	Purpose
Frontend framework	React 18+ (or migrate from vanilla JS)	Component-based, stateful UI
Frontend meta-framework	Next.js 14+ (App Router)	SSR/SSG, routing, API co-location
Backend runtime	Node.js 20 LTS	Server-side JavaScript execution
Backend framework	Express.js (or Fastify)	REST API routing, middleware
Database	PostgreSQL 15+	Relational storage for users, transactions, loans
ORM	Prisma	Type-safe DB access, migrations, relations
Authentication	JWT (access + refresh tokens) or NextAuth.js	Secure, stateless session handling
Password hashing	bcrypt / argon2	Secure credential storage
Validation	Zod	Request/response schema validation
Caching	Redis	Cache external API responses, rate limiting
External data	Alpha Vantage / Twelve Data / RBI open data / a finance data API	Market indices, inflation, interest rates
AI provider	Anthropic Claude API / OpenAI API	Financial chat, initial insight generation
API documentation	OpenAPI/Swagger	Documented, testable API contracts

4.3 Stage 3 — Personal CFO Technologies

Layer	Technology	Purpose
AI orchestration	AI Agent with tool/function calling (Claude/OpenAI function calling)	Structured, grounded reasoning
Agent framework (optional)	LangChain.js / Vercel AI SDK	Tool registration, streaming, memory
Scenario engine	Custom financial modelling library (TypeScript)	Conservative/base/optimistic projections
Goal engine	Custom module + PostgreSQL goals table	Purchase/vacation/retirement planning
Investment analytics	Custom module (portfolio math: CAGR, volatility, allocation)	Portfolio risk/return analysis
Notifications	Email (Resend/SendGrid) + in-app notification table	Proactive financial alerts
Background jobs	BullMQ (Redis-backed queue) or cron (node-cron)	Scheduled insight generation, data refresh
Observability	Sentry (errors) + simple logging (pino)	Production monitoring

4.4 Cross-Cutting / DevOps (applies from Stage 2 onward)

Concern	Technology
Hosting (frontend)	Vercel
Hosting (backend/API)	Render / Railway / AWS (ECS or EC2)
Database hosting	Supabase / Neon / AWS RDS (PostgreSQL)
CI/CD	GitHub Actions (lint, typecheck, test, build, deploy)
Environment config	.env files + hosting-provider secret managers
Testing	Vitest/Jest (unit), Playwright (E2E)
Containerization (optional)	Docker + docker-compose for local dev parity

5. Detailed Feature Specification

5.1 Stage 1 Features (Foundation)

5.1.1 Landing Page

- Hero section: product name, tagline, “Get Started” / “Login” CTAs.
- Responsive nav bar.
- Simple footer.

5.1.2 Authentication UI (frontend-only prototype at this stage)

- `register.html` / `login.html` (or React equivalents) with client-side validation only.
- Fields: Name, Email, Password, Confirm Password.
- No real backend auth yet — this UI is wired to real auth in Stage 2.

5.1.3 Dashboard

- Summary cards: Monthly Income, Monthly Expenses, Savings, Savings Rate, Loan Balance.
- Charts: Income vs Expense (bar), Expense Breakdown (doughnut/pie), Savings Trend (line).
- Recent Transactions list (last 5–10 entries).

5.1.4 Income Management

- Add/edit/delete income entries.
- Fields: Source (Salary, Rental, Business, Freelance, Investment, Bonus, Other), Amount, Date, Description.

5.1.5 Expense Management

- Add/edit/delete expense entries.
- Categories: Housing, Food, Transportation, Shopping, Entertainment, Education, Healthcare, Bills, Subscriptions, Travel, Other.
- Fields: Expense name, Category, Amount, Date.

5.1.6 Loan/EMI Management

- Fields: Loan Name, Loan Type, Principal, Interest Rate, EMI, Remaining Amount, Start Date, End Date.
- Auto-calculated: remaining balance %, loan progress %.

5.1.7 Financial Calculations (pure JS functions)

```
savings = income - expenses
savingsRate = (savings / income) * 100
categoryPercentage = (categoryExpense / totalExpense) * 100
loanProgress = (paidAmount / originalLoanAmount) * 100
financialHealthScore = weighted(savingsRate, expenseControl, debtBurden) // rule-based, not AI
```

5.1.8 Data Model (Stage 1 — client-side only)

```
// Stored in localStorage as JSON
{
  income: [{ id, source, amount, date, description }],
```

```

    expenses: [{ id, name, category, amount, date }],
    loans: [{ id, name, type, principal, interestRate, emi, remaining, startDate, endDate }]
  }

```

5.2 5.2 Stage 2 Features (Intelligence)

5.2.1 5.2.1 Real Authentication

- Registration with email + password (hashed via bcrypt/argon2).
- Login issuing a short-lived access token (JWT) + long-lived refresh token (httpOnly cookie).
- Middleware verifying JWT on every protected route.
- Logout invalidates refresh token (stored/blacklisted server-side or in Redis).

5.2.2 5.2.2 Database Schema (PostgreSQL via Prisma)

```

model User {
  id          String    @id @default(cuid())
  name        String
  email        String    @unique
  passwordHash String
  createdAt    DateTime  @default(now())
  incomes      Income[]
  expenses     Expense[]
  loans        Loan[]
  goals        Goal[]
  investments  Investment[]
  insights     Insight[]
}

model Income {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  source      String
  amount      Decimal
  date        DateTime
  description  String?
  createdAt    DateTime  @default(now())
}

model Expense {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  name        String
  category    String
  amount      Decimal
  date        DateTime
  createdAt    DateTime  @default(now())
}

model Loan {
  id          String    @id @default(cuid())
  userId      String

```

```

    user      User      @relation(fields: [userId], references: [id])
    name      String
    type      String
    principal  Decimal
    interestRate Decimal
    emi       Decimal
    remaining  Decimal
    startDate DateTime
    endDate   DateTime
  }

  model Goal {
    id          String  @id @default(cuid())
    userId      String
    user        User    @relation(fields: [userId], references: [id])
    type        String  // purchase | vacation | retirement | custom
    title       String
    targetAmount Decimal
    targetDate  DateTime
    currentSaved Decimal @default(0)
    createdAt   DateTime @default(now())
  }

  model Investment {
    id          String  @id @default(cuid())
    userId      String
    user        User    @relation(fields: [userId], references: [id])
    type        String  // stock | mutual_fund | etf | gold | fixed_income | other
    name        String
    investedAmount Decimal
    currentValue Decimal
    purchaseDate DateTime
  }

  model Insight {
    id          String  @id @default(cuid())
    userId      String
    user        User    @relation(fields: [userId], references: [id])
    message     String
    category    String  // spending | savings | goal | debt
    createdAt   DateTime @default(now())
    seen        Boolean  @default(false)
  }

```

5.2.3 REST API Specification

```

Auth
POST  /api/auth/register      { name, email, password }
POST  /api/auth/login         { email, password } -> { accessToken }
POST  /api/auth/refresh       (httpOnly cookie) -> { accessToken }
POST  /api/auth/logout

Income
GET    /api/income
POST   /api/income           { source, amount, date, description }

```

```
PUT    /api/income/:id
DELETE /api/income/:id
```

Expenses

```
GET    /api/expenses
POST   /api/expenses          { name, category, amount, date }
PUT    /api/expenses/:id
DELETE /api/expenses/:id
```

Loans

```
GET    /api/loans
POST   /api/loans             { name, type, principal, interestRate, emi, remaining, startDate, ... }
PUT    /api/loans/:id
DELETE /api/loans/:id
```

Analytics

```
GET    /api/analytics/summary      -> { income, expenses, savings, savingsRate, loanBalance }
GET    /api/analytics/spending-trends -> per-category month-over-month trend
GET    /api/analytics/financial-health -> rule-based/AI-assisted score
```

External Data

```
GET    /api/market/indices
GET    /api/market/inflation
GET    /api/market/interest-rates
```

AI (Stage 2 – basic chat, not yet tool-calling agent)

```
POST   /api/ai/chat           { question } -> { answer } // backend gathers data first, then calls AI
```

Every route above (except /api/auth/*) requires a valid Authorization: Bearer <accessToken> header, and every query is scoped with WHERE userId = req.user.id at the database layer — never trust a client-supplied user ID.

5.2.4 5.2.4 AI Financial Chat — Data-Grounding Pattern

User question



Backend authenticates user, loads:

- current + previous month income/expenses
- relevant category breakdowns



Backend computes deltas/percentages (never lets the AI compute these)



Backend sends a structured prompt to the AI model:

```
"User's savings dropped from Rs. 50,000 to Rs. 38,000 (-24%).
Food expense rose from Rs. 7,000 to Rs. 11,000 (+57%).
Explain this to the user in plain language and suggest 1-2
concrete, non-judgmental adjustments."
```



AI response returned to user

5.2.5 5.2.5 Spending Pattern Analytics

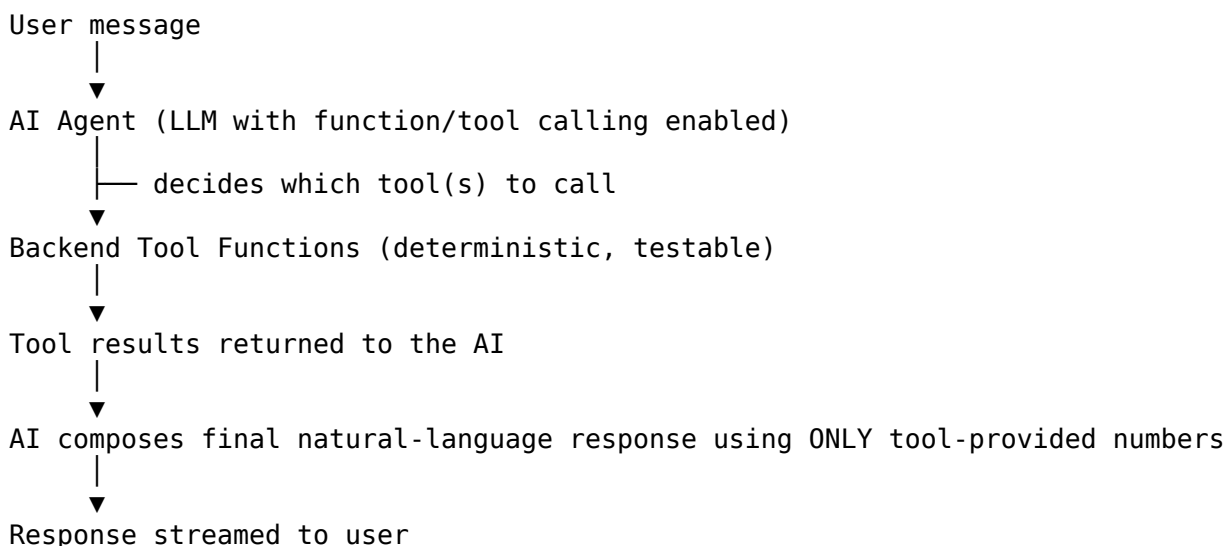
- Month-over-month trend per category.
- Highest spending category detection.
- Recurring vs. one-off expense detection (simple heuristic: same category+amount range appears in ≥ 3 consecutive months).
- Flag categories with $>20\%$ month-over-month increase.

5.2.6 5.2.6 External Financial Data Integration

- Scheduled job (daily) fetches: major market index levels, inflation rate, benchmark interest rate.
- Data normalized and stored in a MarketSnapshot table, cached in Redis for 24h to avoid rate-limit issues.
- Used later (Stage 3) as context for investment/scenario analysis.

5.3 5.3 Stage 3 Features (AI Personal CFO)

5.3.1 5.3.1 AI Agent Architecture



5.3.2 5.3.2 Tool Definitions (function-calling schema, conceptual)

```

getIncome(userId): Income[]
getExpenses(userId, { from, to }): Expense[]
getLoans(userId): Loan[]
getFinancialSummary(userId): { income, expenses, savings, savingsRate, loanBalance }

calculateSavings(income, expenses): number
calculateDebtRatio(totalEMI, income): number // EMI / income
calculateFutureValue(monthlyContribution, annualRatePct, years): number
calculateGoalGap(targetAmount, currentSaved, monthlySavingsCapacity, monthsRemaining): { onT

getMarketData(indicator: "index" | "inflation" | "interestRate"): MarketSnapshot
runScenario(monthlyContribution, years, scenario: "conservative" | "base" | "optimistic"): Scena
  
```

Each tool is a plain, unit-testable TypeScript function with no LLM dependency — the LLM only decides *when* to call them and how to phrase the final answer.

5.3.3 5.3.3 Goal Planning Module

Input (from user, via UI or chat):

```
goalType: "purchase" | "vacation" | "retirement"
targetAmount: number
targetDate: Date
```

Backend computation:

```
monthsRemaining = monthsBetween(today, targetDate)
requiredMonthlySavings = (targetAmount - currentSaved) / monthsRemaining
currentSavingsCapacity = averageMonthlySavings(last 3 months)
gap = requiredMonthlySavings - currentSavingsCapacity
onTrack = gap <= 0
```

Output to user: required monthly savings, current capacity, gap, and (if onTrack: false) suggested category-level cuts based on discretionary vs. essential classification.

5.3.4 5.3.4 Purchase Planner

Same as Goal Planning, with an added **inflation-adjustment** step:

```
inflatedTargetPrice = currentPrice * (1 + assumedInflationRate) ^ yearsUntilPurchase
```

5.3.5 5.3.5 Vacation Planner

Adds an **affordability gate**:

```
if (requiredMonthlySavings + existingEMI) / income > safeDebtServiceRatio:
    flag "This plan may strain your monthly budget"
```

5.3.6 5.3.6 Credit-Trap Prevention

```
newDebtServiceRatio = (existingEMI + proposedNewEMI) / income
if newDebtServiceRatio > 0.40: // configurable threshold
    warn("This purchase may significantly increase your debt burden")
```

This rule fires **before** any AI commentary — it is a hard, deterministic guardrail, not something the LLM can be talked out of.

5.3.7 5.3.7 Investment Portfolio & Analysis

- CRUD for investments (stock, mutual fund, ETF, gold, fixed income, other).
- Computed: total invested, current value, absolute return %, allocation-by-type %.
- AI commentary on concentration risk, diversification, and alignment with stated goals — explicitly caveated as **not financial advice** and **not a return guarantee**.

5.3.8 5.3.8 Scenario Engine

For any recurring contribution:

```
conservativeReturn = 6% p.a.
baseReturn         = 10% p.a.
optimisticReturn   = 14% p.a.
```

```
futureValue(monthlyContribution, annualRate, years) =
    monthlyContribution * (((1 + monthlyRate)^months - 1) / monthlyRate) * (1 + monthlyRate)
```

Output as three side-by-side projections, never a single “guaranteed” number.

5.3.9 5.3.9 Retirement Planner

Inputs: currentAge, retirementAge, currentSavings, monthlyInvestment, expectedReturn, inflationRate
 $\text{requiredCorpus} = \text{estimatedAnnualExpensesAtRetirement} * \text{yearsInRetirement}$ (with withdrawal-rate assumption)
 $\text{projectedCorpus} = \text{futureValue}(\text{currentSavings}, \text{monthlyInvestment}, \text{expectedReturn}, \text{yearsToRetirement})$
 $\text{gap} = \text{requiredCorpus} - \text{projectedCorpus}$

5.3.10 5.3.10 Continuous AI Insights (background job)

Runs daily/weekly via a queue (BullMQ) or cron: - Compare current month vs. 3-month rolling average per category → flag >20% deviations. - Compare goal trajectory vs. target date → flag if falling behind. - Compare savings rate trend → flag sustained decline over 2+ months. - Write results to Insight table; surfaced in-app and optionally via email.

6. Non-Functional Requirements

Category	Requirement
Security	Passwords hashed (bcrypt/argon2, never plaintext); JWTs short-lived; refresh tokens httpOnly + secure cookies; all API routes authenticated except /auth/*; input validated with Zod on every route; SQL injection prevented via Prisma parameterized queries; rate limiting on auth and AI endpoints.
Privacy	Financial data never sent to third parties beyond the chosen AI provider API call itself; AI provider calls should avoid sending raw PII where possible (use IDs/aggregates instead of names/emails in prompts).
Performance	API responses < 300ms for CRUD; dashboard initial load < 2s; external data cached (Redis, 24h TTL) to avoid third-party rate limits and latency.
Reliability	All financial calculations must be pure, deterministic functions with unit tests; AI must never be the sole source of a numeric fact.
Accessibility	WCAG AA color contrast; all interactive elements keyboard-navigable; forms have proper labels.
Responsiveness	Fully usable on mobile (360px+), tablet, and desktop breakpoints.
Scalability	Stateless API design allows horizontal scaling; DB indexes on userId + date columns for all transactional tables.
Observability	Structured logging (pino) in backend; error tracking (Sentry) in both frontend and backend.
Testability	Financial engine: 100% unit-test coverage target; API layer: integration tests per route; critical user flows: Playwright E2E tests.

7. Repository & Project Structure (Recommended)

```
finpilot-ai/
├── apps/
│   ├── web/                                # Next.js frontend
│   │   ├── app/
│   │   │   ├── (auth)/login/
│   │   │   ├── (auth)/register/
│   │   │   ├── dashboard/
│   │   │   ├── goals/
│   │   │   ├── investments/
│   │   │   └── chat/
│   │   ├── components/
│   │   ├── lib/api-client.ts
│   │   └── styles/
│   └── api/                                # Express/Node backend
│       ├── src/
│       │   ├── routes/
│       │   │   ├── auth.routes.ts
│       │   │   ├── income.routes.ts
│       │   │   ├── expenses.routes.ts
│       │   │   ├── loans.routes.ts
│       │   │   ├── analytics.routes.ts
│       │   │   ├── goals.routes.ts
│       │   │   ├── investments.routes.ts
│       │   │   └── ai.routes.ts
│       │   ├── services/
│       │   │   ├── financial-engine.ts    # pure calculation functions
│       │   │   ├── ai-agent.ts           # tool registration + LLM calls
│       │   │   ├── scenario-engine.ts
│       │   │   ├── external-data.service.ts
│       │   │   └── insights.service.ts
│       │   ├── middleware/
│       │   │   ├── auth.middleware.ts
│       │   │   └── validate.middleware.ts
│       │   ├── jobs/
│       │   │   ├── refresh-market-data.job.ts
│       │   │   └── generate-insights.job.ts
│       │   └── server.ts
│       └── prisma/
│           └── schema.prisma
├── packages/
│   └── shared-types/                        # shared TS types between web & api
├── .github/workflows/ci.yml
├── docker-compose.yml
└── README.md
```

8. Build Roadmap & Milestones

Stage	Timeline	Key Deliverable
Stage 1 — Foundation	Now → 1 Sept 2026	Working finance-tracker website (HTML/CSS/JS + LocalStorage)
Stage 2 — Intelligence	Sept 2026 → Dec 2026	Multi-user app with real backend, DB, auth, external data, basic AI chat
Stage 3 — Personal CFO	Dec 2026 → Mar 2027	Full AI agent, goal/investment/scenario/retirement planning, proactive insights

8.0.1 8.1 Suggested Two-Week Sprints (Stage 1 example)

- Sprint 1: Landing page, layout shell, nav, static dashboard skeleton.
- Sprint 2: Income + Expense CRUD wired to LocalStorage.
- Sprint 3: Loan/EMI module + financial calculations.
- Sprint 4: Chart.js integration + financial health score + responsive polish.

(Stages 2 and 3 should be broken down the same way once Stage 1 is demoed and locked.)

9. Instructions for AI-Assisted Development (e.g., Cursor)

If this document is provided to an AI coding assistant to scaffold the project, the assistant should:

1. **Start with Stage 1 only.** Do not scaffold a backend, database, or AI integration until Stage 1's frontend is complete and explicitly approved.
 2. **Use the exact data model in §5.1.8** for `localStorage` in Stage 1, and the exact Prisma schema in §5.2.2 when migrating to Stage 2 — do not invent a different schema.
 3. **Implement the API routes exactly as specified in §5.2.3** — method, path, and payload shape should match, so the frontend contract doesn't need rework later.
 4. **Never let the AI model compute financial numbers directly.** All numeric values shown to the AI (and by the AI to the user) must originate from the pure functions in §5.1.7 (Stage 1) or the tool functions in §5.3.2 (Stage 3).
 5. **Write the financial engine as pure, dependency-free TypeScript functions first**, with unit tests, before wiring them into any API route or AI tool.
 6. **Enforce per-user data isolation server-side** on every query — never rely on the frontend to filter by user.
 7. **Follow the folder structure in §7** so that later stages (adding apps/api, packages/shared-types) don't require restructuring already-written Stage 1 code.
 8. **Ask for confirmation before starting a new stage** — each stage in this document is independently demoable and should be reviewed before the next begins.
-

10 10. Appendix — Glossary of Key Terms

Term	Meaning
EMI	Equated Monthly Installment — fixed monthly loan repayment amount.
CAGR	Compound Annual Growth Rate — a smoothed annual growth rate over a period.
Savings Rate	Percentage of income not spent: $(\text{Income} - \text{Expenses}) / \text{Income} \times 100$.
Debt-to-Income (Debt Service) Ratio	Total EMI obligations divided by income; used to flag over-leveraging.
Tool/Function Calling	An LLM capability where the model can invoke predefined backend functions to fetch real data instead of guessing.
Scenario Modelling	Presenting multiple plausible outcomes (e.g. conservative/base/optimistic) instead of a single predicted number.
Row-Level Authorization	Ensuring database queries are always scoped to the requesting user's own rows, enforced at the query layer.

End of specification. This document is intended to be handed to a development team or an AI coding assistant as the single source of truth for building FINPILOT AI from Stage 1 through Stage 3.